

Tóm tắt toàn khoá

Mục lục

Tóm tắt toàn khoá Software Security	1
Lecture 1: Vocabulary và Vulnerabilities (4 bài)	1
Lecture 2: Formal Verification và BMC (2 bài)	3
Lecture 3: Static Analysis I (BMC + SMT chi tiết, 7 bài)	4
Lecture 4: Static Analysis II - Concurrency (7 bài)	6
Lecture 5: Dynamic Analysis (8 bài)	8
Lecture 6: Case Study (5 bài)	9
Lecture 7: Topics Bổ sung (5 bài)	10
Lecture 8: Code Analysis (6 bài)	11
Cheat sheet 1 trang	12
10 common pitfall	13
Câu hỏi tự kiểm tra	14
Tài liệu tham khảo và bước tiếp	15
Lời cuối	16

Tóm tắt toàn khoá Software Security

Hướng dẫn dùng: tài liệu này tóm tắt **toàn bộ 8 cụm bài giảng** trong 1 file đọc thẳng, phù hợp cho **quick-review** hay **on-call cheat sheet**. Ưu tiên các điểm cốt lõi: định nghĩa chuẩn, công thức ngắn, algorithm step-by-step, common pitfall. Mỗi section dài 5-10 phút đọc. Tổng thời gian: 2-3 giờ. Sau khi đọc, làm **Exercise Set 2** và xem **Cross-reference** để fill gap.

Đọc nhanh: chỉ cần **Cheat sheet 1 trang** và 10 common pitfall. Đủ cho code review nhanh hoặc interview prep.

Lecture 1: Vocabulary và Vulnerabilities (4 bài)

Software Security là gì? (bài 1.1)

Định nghĩa: kỹ thuật và phương pháp đảm bảo phần mềm thực hiện đúng chức năng dự kiến **ngay cả khi bị tấn công có chủ đích**.

Khác biệt then chốt:

Khái niệm	Mối đe dọa	Ví dụ
Safety	Lỗi ngẫu nhiên, thiên tai	Bão cháy, lỗi thoát hiểm
Security	Kẻ tấn công có chủ đích	Khoá cửa, mã hoá, kiểm soát truy cập

Software Security ≠ Cryptography: crypto là một công cụ (thuật toán mã hoá, hàm hash, chữ ký số). Software security là phạm trù rộng hơn bao gồm crypto + memory safety + access control + secure coding + threat modeling.

CIA Triad (bài 1.2)

Ba thuộc tính nền tảng:

- **Confidentiality** (bảo mật): chỉ người được phép đọc được data. Vi phạm = data leak.
- **Integrity** (toàn vẹn): data không bị thay đổi trái phép. Vi phạm = tampering. Đảm bảo bằng MAC, digital signature, hash.
- **Availability** (khả dụng): hệ thống dùng được khi cần. Vi phạm = DoS, ransomware.

Thường bổ sung **AAA**: - **Authenticity**: xác thực danh tính. - **Authorization**: kiểm soát quyền. - **Auditing**: ghi log để truy vết.

Non-repudiation (không thể chối bỏ): được đảm bảo bằng digital signature.

MAC vs Digital Signature: | | MAC (HMAC) | Digital Signature (RSA, Ed25519) | |—|—|—| | Key | Symmetric (cùng key) | Asymmetric (sign by private, verify by public) | | Tốc độ | Nhanh | Chậm hơn 100x | | Non-repudiation | Không | Có | | Use case | API auth | Email, phần mềm, blockchain |

Vulnerabilities (bài 1.3) - phải nhớ

6 lớp lỗ hổng kinh điển trong C/C++:

Lớp	Cơ chế	CWE	Tool detect
Buffer Overflow	Ghi ra ngoài buffer (stack/heap)	CWE-121/122	ASan, CBMC —bounds-check, FORTIFY_SOURCE
Integer Overflow	Phép tính vượt giới hạn kiểu	CWE-190	UBSan, CBMC —signed-overflow-check
Race Condition	Hai thread truy cập shared data không sync	CWE-362	TSan, CBMC —pthread
TOCTOU	Check rồi use, attacker swap giữa	CWE-367	Manual review, dùng fstat+fd
Use After Free	Truy cập memory đã free	CWE-416	ASan, smart pointer (C++)
Format String	printf(user_input) thay vì printf("%s", ...)	CWE-134	Compiler -Wformat-security

Buffer overflow stack layout (phải vẽ được):

[buf[0..n-1]] [saved RBP] [return address]
 □ overflow ghi từ buf, lan đề RBP → return → arbitrary jump

Fix patterns (1 dòng mỗi cái): - gets □ fgets(buf, sizeof(buf), stdin) - sprintf □ snprintf(buf, sizeof(buf), ...) - strcpy □ strncpy (BSD) hoặc snprintf - printf(user) □ printf("%s", user) - n * size □ __builtin_mul_overflow(n, size, &total) hoặc calloc(n, size) - Sau free(p) □ p = NULL; (chống double-free + UAF)

Web Vulnerabilities (bài 1.4) - phải nhớ

6 lớp Web vuln:

Lớp	Cơ chế	Defense
SQL Injection	'OR 1=1--' chèn vào query string	Parameterized query (prepared statement)
XSS (Cross-site Scripting)	<script> chèn vào HTML response	HTML escape, CSP header
CSRF	Browser auto-send cookie cho request từ site khác	CSRF token, SameSite cookie
XXE (XML External Entity)	XML parser load file system	Disable DTD trong parser
SSRF (Server-Side Request Forgery)	Server fetch URL do attacker chọn	Whitelist domain, block AWS metadata IP
ReDoS (Regex DoS)	Regex backtrack exponential	Dùng RE2, RE non-greedy, timeout

Pattern chung: input không tin cậy → ứng dụng → tạo chuỗi → interpreter (SQL, HTML, regex, XML, URL). **Chữ “trộn” sai gây bug.** Fix: tách data và code (parameterize), validate, sanitize.

Lecture 2: Formal Verification và BMC (2 bài)

Formal Verification (bài 2.1) - trọng tâm

Định nghĩa: dùng toán học để **chứng minh** program thoả property, không chỉ test.

Vì sao testing không đủ (câu Dijkstra 1972): > “Program testing can be used to show the presence of bugs, but never to show their absence.”

Hàm cộng 2 int 32-bit có 2^{64} input □ không test hết được.

Property có 2 loại:

Loại	Ví dụ	Counterexample
Safety (“bad never happens”)	“Không có buffer overflow”, “Không deadlock”	Finite trace dẫn tới bad state
Liveness (“good will happen”)	“Mọi request được serve”, “Termination”	Infinite trace ko đạt good state

Soundness vs Completeness:

Tool	Sound (no false negative)	Complete (no false positive)
Tool sound (CBMC trong bound, AFR Coq)	Bug detected □ bug thực	Có thể miss bug
Tool complete (random test)	Có thể miss bug	Bug detected □ bug thực

Lý tưởng: cả 2. Thực tế: trade-off. Most BMC = sound trong bound k.

Định lý Rice: mọi non-trivial property của Turing-complete program đều **undecidable**. Tool phải approximate.

3 họ kỹ thuật:

1. **Model Checking:** tự động khám phá state space. Tool: SPIN, NuSMV, CBMC.
2. **Theorem Proving:** human viết proof, prover check. Tool: Coq, Isabelle, Lean.
3. **Abstract Interpretation:** tính over-approximate property. Tool: Astrée, Frama-C.

	Model Checking	Theorem Proving	Abstract Interp.
Tự động	□	□ (human-aided)	□
Scale	Limited (state explosion)	Bất kỳ	Tốt
Precision	Cao	Tuyệt đối	Approximate

BMC + SMT (bài 2.2) - trọng tâm

BMC formula:

$$\Phi_k = I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge \bigvee_{i=0}^k \neg P(s_i)$$

Đọc: Φ_k **SAT** □ tồn tại execution dài $\leq k$ bước vi phạm property P .

Workflow BMC: 1. **SSA conversion:** mỗi biến gán 1 lần. $x = x + 1 \square x_1 = x_0 + 1$. 2. **Loop unwinding:** while (c) { ... } mở thành if-then-else k tầng. 3. **Encode SMT:** từng instruction □ constraint trong theory phù hợp (BitVec, Array, FP). 4. **Negate assertion:** assertion $a \square$ constraint NOT a . 5. **Solver:** Z3 / cvc5 / MathSAT □ SAT (có counterexample) hoặc UNSAT (an toàn trong bound).

Ví dụ kinh điển hàm $\text{abs}(x)$:

```
int abs(int x) {
    if (x < 0) return -x;
    return x;
}
assert(abs(x) >= 0);
```

- Theory Int (toán học): $-x \geq 0$ luôn đúng □ UNSAT, an toàn.
- Theory BitVec 32: $x = \text{INT_MIN} = -2^{31}$. $-x = \text{INT_MIN}$ (overflow). Assert fail □ **SAT**, counterexample $x = \text{INT_MIN}$.

Bài học: chọn theory đúng = correctness. C semantic dùng BitVec, không Int.

Lecture 3: Static Analysis I (BMC + SMT chi tiết, 7 bài)

V&V (bài 3.2)

Verification vs Validation: - V: “are we building the product right?” - khớp spec không. - Val: “are we building the right product?” - đáp ứng nhu cầu user không.

V-model: mỗi level (requirements / design / module / integration / system / UAT) có level verification tương ứng.

Static vs Dynamic verification: - Static: không chạy code. SAST, BMC. Sound (đôi khi), nhiều false positive. - Dynamic: chạy code. Test, fuzz, ASan. False positive thấp, false negative cao.

State space exploration (bài 3.3)

State explosion: số state $\approx |V_1| \times |V_2| \times \dots \times |V_n|$. Với 10 int 32-bit, state space = $(2^{32})^{10} = 2^{320}$, vô nghĩa enumerate.

Symbolic vs explicit: - Explicit: lưu từng state, hash table. Tool: SPIN. - Symbolic: biểu diễn tập state bằng công thức (BDD, SMT). Tool: NuSMV, CBMC.

Symbolic execution: chạy chương trình với input symbolic, fork tại mỗi branch, mỗi path tích lũy **path constraint**.

SAT và DPLL (bài 3.4) - phải nhớ

SAT problem: cho CNF formula ϕ trên n biến boolean, tồn tại gán giá trị thoả ϕ không?

CNF: $(l_{11} \vee l_{12} \vee \dots) \wedge (l_{21} \vee \dots) \wedge \dots$ — conjunction of disjunctions.

DPLL algorithm:

```
DPLL( $\Sigma$ ):
  if  $\Sigma$  empty: return SAT
  if  $\Sigma$  has empty clause: return UNSAT
   $\Sigma$  = UnitPropagate( $\Sigma$ )           // clause 1 literal  $\rightarrow$  assign
   $\Sigma$  = PureLiteralElim( $\Sigma$ )      // literal chỉ xuất hiện 1 polarity  $\rightarrow$  assign
  if  $\Sigma$  done: handle
   $l$  = ChooseLiteral( $\Sigma$ )
  return DPLL( $\Sigma \wedge l$ ) OR DPLL( $\Sigma \wedge \neg l$ )
```

CDCL (modern, in Z3/MiniSat): DPLL + **conflict-driven clause learning**: 1. Decide Σ propagate Σ conflict. 2. Analyze conflict, learn clause để tránh lặp. 3. Non-chronological backjumping (jump back nhiều level). 4. Restart heuristic, VSIDS variable ordering.

Heuristics quan trọng: - **VSIDS** (Variable State Independent Decaying Sum): chọn biến xuất hiện nhiều trong learned clause gần đây. - **Unit propagation**: khi clause còn 1 literal chưa assigned, force assign.

SMT theories (bài 3.5) - phải nhớ

SMT = SAT + Theory:

Theory	Symbol	Decidable?	Tool
EUF (Uninterpreted Function)	$f(a) = f(b), a = b \square$ $f(a) = f(b)$	Decidable	Z3
LIA (Linear Integer Arithmetic)	$2x + 3y \leq 10$	Decidable	Z3
LRA (Linear Real)	$2.5x + y \leq 1$	Decidable	Z3
NIA / NRA (Non-linear)	$x^2 + y^2 = 1$	Semi-decidable	Z3 incomplete
BV (BitVector)	C int 32-bit, bitwise op	Decidable (NP-complete)	Z3, Boolector
Array	select(store(a, i, v), i) = v	Decidable	Z3

Theory	Symbol	Decidable?	Tool
FP (Floating Point)	IEEE 754	Decidable (đắt)	Z3, MathSAT

DPLL(T): SAT solver chạy DPLL, gọi theory solver định kỳ để check theory consistency.

Nelson-Oppen: combine 2 theory disjoint signature qua “purification” + “equality propagation”. Cho phép viết formula mix EUF + LIA, etc.

Encoding numbers and floats (bài 3.6)

Integer encoding: - `int C` $\square$ `BitVec 32`. Operation `bvadd`, `bvmul`, `bvslt` (signed), `bvult` (unsigned). - Signed: two’s complement. `INT_MIN` = -2^{31} , `INT_MAX` = $2^{31} - 1$. - Overflow: wrap-around (unsigned) hoặc UB (signed C standard, nhưng compilers tend to wrap).

Float encoding (IEEE 754): - `float` = 32 bit: 1 sign + 8 exponent + 23 mantissa. - `double` = 64 bit: 1 + 11 + 52. - Special: $+0$, -0 , $+\infty$, $-\infty$, NaN. - $0.1 + 0.2 \neq 0.3$ vì không biểu diễn chính xác trong nhị phân. - NaN $\neq$ NaN (theo design). Check NaN: `isnan(x)`.

Rounding mode: round-to-nearest-even (default), toward zero, toward $+\infty$, toward $-\infty$.

Encoding pointers and memory (bài 3.7)

Pointer encoding (CBMC): - `Pointer = (object_id, offset)`. - `p = malloc(100)`: `object_id` mới, size 100. - `p[i]`: read at (`object_id(p)`, `offset(p) + i * sizeof(elem)`). - Bounds check: `offset < object_size`.

Memory model: - `--mm fixed`: object có địa chỉ cố định. - `--mm align`: object align theo type (default). - `--mm offset`: track offset chính xác.

Array theory axiom: - `select(store(a, i, v), j) = v` nếu $i = j$. - `select(store(a, i, v), j) = select(a, j)` nếu $i \neq j$.

Lecture 4: Static Analysis II - Concurrency (7 bài)

Loop Unwinding + k-induction (bài 4.2)

Unwinding:

`while (cond) body`

mở thành:

```
if (cond) { body
  if (cond) { body
    ... // k tầng
  }
}
```

Unwinding assertion: thêm `assert(!cond)` sau k tầng để báo nếu cần unwind sâu hơn.

k-induction: chứng minh: 1. Base case: property hold ở step 0..k. 2. Inductive step: nếu hold ở $i..i+k-1$ thì hold ở $i+k$.

Cho ra proof cho **mọi depth**, không bị limit. Triển khai trong CBMC, ESBMC.

Bit-blasting + Arrays (bài 4.3)

Bit-blasting: convert BitVec operation thành SAT (boolean) bằng cách “phẳng” mỗi bit. bvadd 32-bit = full adder circuit với ~100 boolean variable + 100 clause.

Lambda-term cho array: $\text{select}(\text{store}(a, i, v), j) \equiv \text{ite}(i = j, v, \text{select}(a, j))$. SMT solver dùng eager hoặc lazy.

Concurrency Verification (bài 4.4) - trọng tâm

Race condition: hai thread truy cập cùng memory, ít nhất 1 write, không sync.

Atomicity violation: code expect atomic nhưng thực tế không. Ví dụ:

```
if (balance >= amount) {           // check
    balance -= amount;              // use
}
// Hai thread cùng pass check, balance âm.
```

Deadlock (4 điều kiện Coffman 1971): 1. Mutual exclusion. 2. Hold and wait. 3. No preemption. 4. Circular wait.

Memory model:

Model	Đặc tính	CPU
SC (Sequential Consistency)	Mọi thread thấy cùng total order	Lý tưởng, không tồn tại trên CPU thực
TSO (Total Store Order)	Store-load reorder cho phép (do store buffer)	x86, x86-64
Weak	Mọi reorder cho phép, cần fence	ARM, POWER

Happens-before: - Cùng thread: program order. - Synchronize: $\text{unlock}(L)$ happens-before $\text{lock}(L)$ tiếp theo. - Transitive.

Hai access **không happens-before** nhau và ≥ 1 write = data race.

CBA (bài 4.5) - trọng tâm

Qadeer-Rehof observation: hầu hết bug concurrency xảy ra với **rất ít context switch** (2-3, không hàng nghìn).

Context-bounded analysis: chỉ explore schedule có $\leq K$ context switch. $K=2$ đã catch ~70% bug.

Số schedule với CBA: - Round-robin, K rounds, n threads, m instruction/thread: schedule scale **polynomial** trong K (không exponential).

Hoà giải state explosion: từ $\binom{nm}{m, m, \dots}$ (factorial) $\square \sim O((nm)^K)$ (polynomial).

Lazy vs Schedule recording (bài 4.6)

- **Eager (depth-first):** explore mỗi schedule từ đầu đến cuối. Cây quyết định exponential.
- **Lazy interleaving:** trì hoãn quyết định context switch đến khi gặp shared access.
- **Schedule recording:** chạy 1 schedule, ghi lại, replay với variation.

Mazurkiewicz trace: hai schedule “equivalent” nếu chỉ khác thứ tự các operation **không phụ thuộc** (cùng thread hoặc khác variable). Khai thác để giảm số schedule.

DPOR (Dynamic Partial-Order Reduction): chỉ explore một schedule từ mỗi equivalence class.

Sequentialization (bài 4.7)

KISS (Keep It Simple, Stupid): biến multi-thread thành **sequential** bằng cách lặp round-robin K lần. Mỗi round: T1 chạy, T2 chạy, ..., Tn chạy. Reuse sequential BMC tool.

LR scheme: recursion-based, mỗi context switch là 1 recursive call. Linh hoạt hơn nhưng tốn stack.

Lecture 5: Dynamic Analysis (8 bài)

Security Testing (bài 5.2)

4 phương pháp: - **Vulnerability scanning**: auto tool tìm CVE/CWE pattern. Nessus, OpenVAS. - **Penetration testing**: manual expert, creative attack. Đắt nhưng sâu. - **Bug bounty**: crowdsource. HackerOne, Bugcrowd. - **Red team**: simulate APT, mọi cách (technical + social engineering).

Coverage Criteria (bài 5.3) - trọng tâm

Criterion	Định nghĩa	Test count
Statement	Mọi statement chạy ≥ 1 lần	Low bar
Branch	Mọi branch true/false taken	Mạnh hơn statement
Condition	Mọi sub-condition true/false	
MC/DC	Mỗi sub-condition độc lập ảnh hưởng outcome	$n + 1$ với n condition
Multiple condition (full)	Mọi tổ hợp boolean	2^n - không scale
Path	Mọi execution path	Exponential, không tractable

MC/DC = chuẩn aerospace (DO-178C), avionics, automotive (ISO 26262).

Monitoring với LTL + Büchi (bài 5.4)

LTL operators: - Xp : next, p ở step tiếp. - Gp : globally, p luôn đúng. - Fp : finally/eventually, có lúc p đúng. - $p U q$: until, p đúng cho tới khi q đúng.

Büchi automaton (BA): accept **infinite word** nếu run đi qua accepting state **vô hạn lần**.

LTL $\square$ BA: mỗi LTL formula convert thành BA. Monitor = mô phỏng BA trên trace runtime.

Safety check: monitor đạt non-accepting sink $\square$ vi phạm. Tractable. **Liveness check**: cần observe infinite trace. Practical: **bounded liveness** (p xảy ra trong k event).

Fuzzing (bài 5.5-5.7) - trọng tâm

Fuzzing: sinh input random/mutated, mục tiêu trigger crash hoặc assertion fail.

3 loại:

Loại	Mechanism	Tool
Black-box random	Random bytes	Trinity
Black-box grammar	Sinh theo grammar (input syntax)	Peach, libProtobufMutator
Black-box mutation	Mutate seed corpus	radamsa
Coverage-guided (grey-box)	Mutate + ưu tiên input mới cover code path mới	AFL , libFuzzer, honggfuzz
White-box symbolic	Symbolic execution, solver tìm input cho path mới	KLEE, SAGE, Driller (hybrid)

AFL workflow: 1. Instrument target (compile-time pass, ghi log mỗi edge basic block). 2. Forkserver: fork process từ saved state, tránh re-exec. 3. Mutation: bit flip, arithmetic, splice 2 seed. 4. Coverage feedback: input mới cover new edge $\square$ add to corpus. 5. Energy schedule: input nhiều cover được mutate nhiều hơn.

DSE (Dynamic Symbolic Execution) trade-off: chính xác hơn coverage-guided cho path có **magic constant** (xác suất random 2^{-32}), nhưng chậm và path explosion.

Driller: combine AFL (mutation) + DSE (khi AFL stuck). Best of both.

BMC for test generation (bài 5.8)

Cách dùng BMC sinh test: 1. Cho property ϕ ta muốn cover (e.g., reach line N). 2. Negate: assert $\neg\phi$. 3. CBMC tìm counterexample = input đạt ϕ . 4. Counterexample = test case.

CBMC có flag `--cover line`, `--cover branch` để auto-generate test.

Lecture 6: Case Study (5 bài)

Methodology chung

6-bước tư vấn security cho dự án: 1. **Hiểu context:** tech stack, scale, compliance, threat model. 2. **Identify asset:** data quan trọng, system critical. 3. **Threat model:** dùng **STRIDE** (Spoofing, Tampering, Repudiation, Info disclosure, DoS, Elevation of privilege). 4. **Risk assess:** **DREAD** scoring, hoặc **CVSS 3.1** (Critical 9-10, High 7-8.9, Medium 4-6.9, Low 0.1-3.9). 5. **Roadmap:** 3 phase Must/Should/Nice-to-have, cost & ROI. 6. **Iterate:** review hằng quý.

4 domain (bài 6.2-6.5)

Domain	Focus	Compliance	Key control
Web/SaaS	OWASP Top 10	SOC 2	Auth0, WAF, Dependabot, pen test
Fintech	Transaction integrity, PII	PCI-DSS, KYC/AML	HSM, signed audit log, 4-eye principle
IoT/Embedded	Memory safety, secure boot	Cyber Resilience Act	Secure boot chain, OTA update signed

Domain	Focus	Compliance	Key control
Enterprise Cloud	Zero Trust, mTLS	ISO 27001, SOC 2	IAM, microsegmentation, SIEM, ransomware 3-2-1 backup

Lecture 7: Topics Bổ sung (5 bài)

Cryptography Basics (bài 7.2) - phải nhớ

Hash: - Properties: pre-image resistance, second pre-image, collision resistance. - **Use:** integrity, fingerprint, NOT password (vì fast). - Algorithms: SHA-256 (256-bit output, secure), SHA-3, BLAKE3. - **MD5, SHA-1 broken:** không dùng cho security.

Password hashing: - bcrypt (cost factor adjustable), argon2 (winner Password Hashing Competition 2015), scrypt. - Default: argon2id, cost ~100ms. - Slow là intentional, tránh brute force.

Symmetric encryption: - **AES-GCM** (128 or 256 bit key): authenticated encryption. **Use this.** - **ChaCha20-Poly1305:** alternative cho ARM (faster without AES-NI). - **Tránh:** AES-ECB (leak pattern), AES-CBC without HMAC (padding oracle).

Asymmetric: - **RSA-2048+:** legacy, dùng cho signing và key exchange. - **Ed25519:** modern signature, nhanh, nhỏ key. - **X25519:** ECDH key exchange (TLS 1.3).

PKI (Public Key Infrastructure): CA, cert chain, OCSP, CRL.

HKDF: derive multiple keys từ 1 secret. Extract (uniform) + Expand (multiple).

OWASP Top 10 (2021) (bài 7.3)

A01-A10: - **A01 Broken Access Control** (#1, ~94% app có vấn đề). - **A02 Cryptographic Failures** (data exposure). - **A03 Injection** (SQLi, command, XSS). - **A04 Insecure Design.** - **A05 Security Misconfiguration.** - **A06 Vulnerable and Outdated Components.** - **A07 Identification and Authentication Failures.** - **A08 Software and Data Integrity Failures** (supply chain). - **A09 Security Logging and Monitoring Failures.** - **A10 SSRF.**

Examples thực tế: - A01: IDOR (Insecure Direct Object Reference) - /api/user/123 cho user khác. - A10: Capital One 2019 SSRF qua AWS metadata endpoint □ 100M record leak. Fix: IMDSv2 token-based.

CBMC Tutorial (bài 7.4) - phải nhớ cmd

Cài đặt: brew install cbmc (macOS), apt install cbmc (Ubuntu).

Basic check:

```
cbmc file.c --bounds-check --pointer-check --signed-overflow-check
```

Flags hay dùng: | Flag | Tác dụng | | | | |
 --unwind N | Loop unwind depth | |
 --unwinding-assertions |
 Báo nếu loop cần > N | |
 --bounds-check | Array OOB | |
 --pointer-check | NULL deref, UAF | |
 --signed-overflow-check | Signed int overflow | |
 --unsigned-overflow-check | Unsigned wrap | |
 --memory-leak-check | Memory leak | |
 --pthread | Concurrency | |
 --cover line/branch | Test gen | |
 --trace | Show counterexample |

__CPROVER_assume(cond): assume cond true (constrain input). __CPROVER_assert(cond, "msg"): assertion.
nondet_int(): any int (symbolic input).

Harness pattern:

```
int main(void) {  
    int x = nondet_int();  
    __CPROVER_assume(x >= 0 && x < 100); // bound input  
    int y = target_function(x);  
    __CPROVER_assert(y >= 0, "y always non-negative");  
    return 0;  
}
```

Secure SDLC (bài 7.5)

Microsoft SDL (12 practice): training, requirements, design, implementation, verification, release, response.

OWASP SAMM v2 (Software Assurance Maturity Model): 5 business function × 3 practice × 3 maturity level (0-3).

DevSecOps = Shift Left: security vào CI/CD từ developer, không chờ tới production.

Pipeline tools: - Pre-commit: gitleaks (secret scan). - PR: SonarCloud (SAST), Snyk/Dependabot (SCA), Trivy (container). - Pre-deploy: Checkov (IaC), zaproxy (DAST). - Production: SIEM (Splunk, Datadog), WAF (CloudFlare).

Lecture 8: Code Analysis (6 bài)

Phương pháp 6 bước

1. **Hiểu intent**: code đang cố làm gì?
2. **Liệt kê assumption**: input bound, pointer non-null, etc.
3. **Checklist bug**: memory / integer / logic / crypto / injection / auth.
4. **CVSS rating**: Critical/High/Med/Low.
5. **Fix**: minimal, correct, commented, tested.
6. **Verify**: tool (CBMC/ASan/etc.) + regression test.

39 code pattern tóm tắt

Cụm 1 (Vulnerabilities): gets(), NULL deref, double free, UAF, integer overflow, format string, race, TOCTOU, signedness, off-by-one.

Cụm 2 (BMC encoding): array bounds nondet, pointer arith OOB, float compare, NaN, SSA, array alias, pointer alias, integer division by zero / INT_MIN/-1.

Cụm 3 (Concurrency): race n++, atomicity withdraw, deadlock, lost wakeup, TSO Dekker, ABA, double-checked locking, false sharing.

Cụm 4 (Testing): coverage demo, MC/DC challenge, fuzz target naïve, DSE magic constant, BMC test gen, parser fuzz.

Exercise Set 2 (7 bài, CWE mapping): | Bài | CWE | Tool | |—|—|—| | sprintf overflow | CWE-121 | ASan
| | calloc int overflow | CWE-190 | UBSan, __builtin_mul_overflow | | uninitialized variable | CWE-457 | MSan,

-Wuninitialized | | off-by-one loop | CWE-193 | CBMC, ASan | | race counter | CWE-362 | TSan | | UAF
linked list | CWE-416 | ASan, smart pointer | | struct padding | ABI | pahole, manual |

Cheat sheet 1 trang

Vocabulary cốt lõi

Term	1-câu
CIA	Confidentiality / Integrity / Availability
Safety vs Security	Random vs adversarial
Sound	Bug found $\square$ bug real (no false negative)
Complete	No false positive
BMC	Bounded Model Checking, k-step formula
SMT	SAT + theory
SAT	Boolean satisfiability, NP-complete
SSA	Static Single Assignment
CDCL	DPLL + conflict learning
EUf, LIA, LRA, BV, FP, Array	SMT theories
BitVec	Theory cho int kích thước cố định
Race	2 thread, ≥ 1 write, no sync
TOCTOU	Check rồi use, attacker swap giữa
UAF	Use After Free
TSO	x86 memory model, store-load reorder
CBA	Context-bounded analysis
DPOR	Dynamic Partial-Order Reduction
Mazurkiewicz trace	Equivalence class of schedules
KISS / LR	Sequentialization schemes
MC/DC	Modified Condition/Decision Coverage
LTL	Linear Temporal Logic
Büchi automaton	Infinite-word acceptor
AFL	Coverage-guided fuzzer
DSE	Dynamic Symbolic Execution
STRIDE	Threat model: Spoof/Tamper/Repudiate/InfoLeak/DoS/Elevate
DREAD	Risk: Damage/Reproducibility/Exploitability/Affected/Discoverability
CVSS	Common Vulnerability Scoring System
OWASP Top 10	10 lớp web vuln (A01-A10)
CWE	Common Weakness Enumeration
PCI-DSS	Payment Card Industry Data Security Standard
Zero Trust	Never trust always verify
HSM	Hardware Security Module
CBMC	Bounded Model Checker for C
ASan / MSan / TSan / UBSan	Address / Memory / Thread / UB Sanitizer
SAST / DAST	Static / Dynamic AppSec Testing
SCA	Software Composition Analysis (dep scan)
SDLC	Software Development Lifecycle
SDL	Microsoft Security Development Lifecycle
SAMM	OWASP Software Assurance Maturity Model

Công thức và số phải nhớ

Thứ	Giá trị
int8_t range	$-128 \dots 127$
uint8_t range	$0 \dots 255$
int32_t range	$-2^{31} \dots 2^{31} - 1 \approx \pm 2.15 \times 10^9$
uint32_t max	$2^{32} - 1 \approx 4.29 \times 10^9$
BMC formula	$I(s_0) \wedge \bigwedge T(s_i, s_{i+1}) \wedge \bigvee \neg P(s_i)$
MC/DC test count	$n + 1$ với n condition
Multiple cond. full	2^n
CBA tradeoff	$O((nm)^K)$ thay $\binom{nm}{m, \dots}$
4 điều kiện deadlock	Mutex / Hold-Wait / No-Preempt / Circular
6 lớp Web vuln	SQLi / XSS / CSRF / XXE / SSRF / ReDoS
Boehm cost ratio	Req:Design:Code:Test:Prod = 1:5:10:50:200
CVSS scale	0.1-3.9 Low / 4-6.9 Med / 7-8.9 High / 9-10 Critical
OWASP A1 (2021)	Broken Access Control
AES-GCM key size	128 hoặc 256 bit
SHA-256 output	256 bit = 32 byte
Ed25519 key size	32 byte (public + private)

Tool stack 1 dòng

Bug class	Best tool
Buffer overflow	ASan + FORTIFY_SOURCE
Integer overflow	UBSan, __builtin_*_overflow
Uninit	MSan, -Wuninitialized
UAF	ASan
Race condition	TSan
Deadlock	TSan, CBMC –deadlock-check
Format string	-Wformat-security
BMC verify	CBMC / ESBMC
Fuzzing parser	libFuzzer + ASan + dictionary
Secret leak	gitleaks pre-commit
Dep vuln	Dependabot, Snyk
Container CVE	Trivy, Snyk Container

10 common pitfall

1. Soundness vs completeness lẫn lộn:

- Sound = “if tool says safe, then safe” (no false negative).
- Complete = “if there’s a bug, tool finds it” — wait, that’s the opposite. Let me restate:
- Tool **sound**: nếu tool báo bug, đó là bug thật. (No FP).
- Tool **complete**: tìm được mọi bug. (No FN).
- Mỗi tool ít nhất là một, hiếm khi cả hai.
- Quy ước trong verification: sound = “không miss bug” = no FN. Complete = “không báo nhầm” = no FP. **Cần kiểm tra context của câu hỏi.**

2. `abs(INT_MIN)` **overflow**: `abs(-231) = 231` không biểu diễn được trong `int32`. Return `INT_MIN` (negative). Bug.
3. `NaN == NaN` **is false**: IEEE 754 design. Dùng `isnan(x)` để check.
4. `memcpy(p, q, len)` **với len âm**: `signed int len = -1` cast `size_t = SIZE_MAX`. `Memcpy` 18 quintillion byte.
5. **MC/DC test count** = $n + 1$, không phải 2^n . Đây là điểm phân biệt với full multiple-condition.
6. **TSO cho phép store-load reorder**: trong Dekker's algorithm, hai thread có thể đọc thấy giá trị stale. Cần `memory_order_seq_cst` hoặc fence.
7. **Double-checked locking bị bug trên weak memory**: cần `atomic + release/acquire`, hoặc `pthread_once`.
8. **Spurious wakeup**: `pthread_cond_wait` có thể return mà không signal. **Luôn dùng while, không if**.
9. **MAC vs Digital Signature**: MAC symmetric (cả 2 bên có cùng key), DS asymmetric (verify by public key). DS cho non-repudiation, MAC không.
10. `malloc(0)` **undefined behavior**: có thể return NULL hoặc unique pointer. Đừng rely vào behavior cụ thể.

Câu hỏi tự kiểm tra

Trả lời được hết = nắm vững toàn bộ khoá.

Phần A: định nghĩa (mỗi câu 1 phút)

1. Phân biệt safety và security qua 1 ví dụ.
2. CIA Triad là gì? Cho ví dụ vi phạm mỗi thuộc tính.
3. Sound vs complete trong context BMC?
4. Định lý Rice nói gì?
5. SSA form là gì? Lý do dùng trong BMC?
6. DPLL vs CDCL khác gì?
7. EUF, LIA, BV: ví dụ mỗi theory.
8. Race condition vs data race khác gì?
9. SC vs TSO vs Weak memory model: 1 example phân biệt.
10. CBA là gì? Số schedule với $K=2$ vs full?
11. MC/DC: tính cho $(A \parallel B) \ \&\& \ C, n + 1 = ?$
12. LTL: phân biệt Gp, Fp, Xp, pUq .
13. AFL workflow 5 bước.
14. DSE vs AFL: khi nào DSE thắng?
15. STRIDE 6 mối đe dọa.
16. OWASP A01 là gì? Ví dụ.
17. AES-GCM vs AES-ECB: tại sao GCM?
18. RSA vs Ed25519: khác biệt thực tế.
19. SDL vs SAMM: scope khác nhau.
20. CBMC harness điển hình 4 thành phần.

Phần B: phân tích code (mỗi câu 3-5 phút)

Cho code, tìm bug + tool detect:

```
char *p = malloc(100);
strcpy(p, user_input);
free(p);
printf("%s", p);
```

```
int total = count * size;
char *buf = malloc(total);
```

```
pthread_mutex_lock(&L);
if (balance >= amount) {
    pthread_mutex_unlock(&L);
    balance -= amount;
}
```

```
double x = 1.0 / 0.0;
if (x == x) printf("OK");
else printf("FAIL");
```

(Đáp án: tham khảo [Lecture 8 sample analysis](#)).

Phần C: tính toán (mỗi câu 2 phút)

1. Encode `int x; if (x > 0) y = 1; else y = 2;` thành SSA + SMT.
2. Tính path constraint cho cả 2 branch của `if (a > b) max = a; else max = b;`
3. Cho `int8_t x = 127; x++;` giá trị `x` mới = ?
4. `(uint8_t)(-1) = ?`
5. `INT_MAX + 1` trong signed C32 (undefined, nhưng x86 wrap to?).
6. MC/DC test count cho decision $(A \vee B) \wedge (C \vee D)$.
7. Hash collision với n bit output: cần thử bao nhiêu input để 50% chance collision (Birthday paradox)?
8. $0.1 + 0.2 - 0.3 = ?$ (theo IEEE 754 double).

(Đáp án: 1+2 xem Lec 3 + 8; 3 = -128 (wrap); 4 = 255; 5 = INT_MIN (typically); 6 = 5 test ($n + 1$ với $n = 4$); 7 = $2^{n/2}$ (birthday); 8 $\approx 5.55 \times 10^{-17}$).

Tài liệu tham khảo và bước tiếp

- **Cross-reference**: tra cứu bài theo prerequisite/used-by.
- **Glossary**: 200+ thuật ngữ alphabetical.
- **DS Perspective**: liên hệ ML/DS analogy.
- **Exercise Set 2**: 7 bài analysis code có đáp án.
- **Lecture 8**: 39 code pattern review chi tiết.

Sách tham khảo: 1. Clarke, Grumberg, Kroening (2018): *Model Checking* 2nd ed. 2. Kroening, Strichman: *Decision Procedures*. 3. Stuttard, Pinto: *The Web Application Hacker's Handbook* (Web vuln). 4. Howard, Lipner: *The Security Development Lifecycle* (SDL). 5. Schneier: *Cryptography Engineering*.

Online: - OWASP Top 10 2021: <https://owasp.org/Top10> - CWE Top 25: <https://cwe.mitre.org/top25> - CVSS calculator: <https://www.first.org/cvss/calculator/3.1> - CBMC manual: <https://www.cprover.org/cbmc/>

Lời cuối

Tài liệu summarize này **không thay thế** việc đọc chi tiết mỗi bài. Nó là **cây xương sống** để bạn nắm bird-eye view và quick-recall các điểm chính. Khi gặp câu hỏi không trả lời được, **quay lại bài chi tiết** trong 8 cụm, đọc kỹ section liên quan.

Trên thực tế công việc, **thực hành đọc code + chạy tool** đáng giá hơn ghi nhớ định nghĩa. Sau khi đọc xong tài liệu này, hãy: 1. Clone 1 dự án open source C/C++. 2. Chạy ASan + UBSan + CBMC. 3. Tìm + report 1 bug thật.

Đó là validation cuối cùng cho mọi kiến thức trong khoá học này.